

didi_workbench_bundle – Setup Guide

This is written for you to follow on Posit Workbench, by hand, with no help from the assistant that built this bundle. Every command below is meant to be typed into a Workbench Terminal (or run as a notebook cell – see `notebook/run_on_workbench.ipynb`) from inside the unzipped `didi_workbench_bundle/` folder.

1. What this test does and does not establish

This bundle generates a **synthetic** raw dataset, calibrated against the real 20% sample’s own diagnostics (`export_20260911/20pct/`, bundled here for you to check), and runs the unchanged `psm_260915.py` estimator against it end to end.

What it establishes, if every stage passes: the estimator’s code runs correctly on data shaped like the real thing – right schema, right row-count quotas, right treated/control funnel structure, and (this is the part Gate 2’s synthetic data got wrong) a realistic matching-load pattern, so that every code branch (panel build, risk-set partitioning, propensity fitting, optimal matching, balance tables, outcome models, longitudinal mediation with bootstrap, MSK dropout sensitivity, lag1-vs-lag3 comparison, figure generation) actually executes, and you get real memory/time measurements for planning the secure-machine run.

What it does NOT establish: real-data overlap, causal identification, propensity model fit stability on the actual data, whether the actual point estimates and confidence intervals behave sensibly, or any actual finding of the paper. Synthetic data can prove code *runs*; it cannot prove results are *right*. The real 20% data has already passed the panel build and matching stages on the secure machine (see `export_20260911/20pct/260909/pipeline.log`) – this bundle exists to de-risk everything *after* that point, at the real matched-sample scale (~115,000 pairs), before spending secure-machine time on it.

2. Uploading the bundle

1. On your laptop, zip the `didi_workbench_bundle/` folder (or use the `didi_workbench_bundle.zip` already produced for you – see `FINAL_REPORT.md` for its size).
2. In Posit Workbench, use the Files pane’s **Upload** button (or `scp/rsync` if you have shell access to the Workbench host directly) to place the zip somewhere in your home directory, e.g. `~/didi_workbench_bundle.zip`.
3. Open a Terminal (see step 3 below for how) and unzip it:

```
cd ~
unzip didi_workbench_bundle.zip
cd didi_workbench_bundle
```

4. Generated data and run outputs are written **next to** the bundle folder, at `../didi_data` relative to wherever you unzipped it (controlled by `DATA_ROOT` in `bundle_settings.env`) – so if the bundle is at `~/didi_workbench_bundle`, data lands at `~/didi_data`. This keeps the bundle folder itself small if you need to re-zip and re-upload it later (e.g. after editing `bundle_settings.env`).
-

3. Starting the right session

1. From the Workbench homepage, start a **new Session**, not a Job, so you get an interactive Terminal.
2. In the session launcher: pick the **largest memory profile you are allowed**. Memory is the binding constraint for this whole exercise (see section 6) – there is no reason to under-request it.

3. In the same launcher, find the **Python version** dropdown and select **3.9.1** specifically, if it is offered. This project is pinned to Python 3.9.1 and Polars 0.20.16 (see `requirements.txt` for why: those two are hard version pins that the rest of the pinned scientific-Python stack was validated against).
 4. If 3.9.1 is not offered: run `ls /opt/python` in a Terminal to see what interpreters Workbench has installed. Any 3.9.x patch is acceptable (this bundle's `scripts/setup_env.sh` will tell you plainly if it found 3.9.1 exactly or a different 3.9.x patch). If no 3.9.* directory exists at all, ask your Workbench administrator to install one – do **not** proceed with a 3.10+ or 3.8- interpreter; the pinned `polars==0.20.16` and `numpy==1.25.0` wheels are tied to the CPython 3.9 ABI and will either fail to install or (worse) silently misbehave.
 5. Once the session is running, open a **Terminal**: in the Jupyter-style launcher, click **Terminal** (not a notebook kernel) – this is the tab you'll spend most of your time in. The notebook route (`notebook/run_on_workbench.ipynb`) is available too, but prefer the Terminal for anything that takes more than a couple of minutes (see section 5 for why).
-

4. Step-by-step commands, in order

Run these from inside `didi_workbench_bundle/`. Each is followed by what a good result looks like, and the corresponding number this bundle measured on the laptop it was built and tested on (see `LOCAL_TEST_REPORT.md` for full detail) as a rough comparison point – Workbench numbers will differ (different CPU, different memory ceiling, different Python 3.9.x patch) and that's expected; wildly different orders of magnitude are the thing to actually worry about.

4.1 `bash scripts/setup_env.sh`

Creates `venv_didi/` with the pinned Python 3.9.x interpreter and installs every pin in `requirements.txt`. **Good result:** ends with an “Environment check” block reporting Python 3.9.x, Polars exactly 0.20.16, `streaming_collect_mode: streaming=True`, and “Pandas/Arrow round-trip: PASSED.” If a pin fails to install, the script stops and names the exact pin – fix the environment (e.g. a missing system library) rather than substituting a different version.

Laptop comparison: Python 3.9.25 (Homebrew, not 3.9.1 – this laptop had no way to install 3.9.1 exactly), Polars 0.20.16 exact match, streaming mode exact match, round-trip PASSED.

4.2 `bash scripts/setup_env.sh --self-test`

Runs the estimator's own built-in `--self-test` (240 synthetic people, full pipeline) under `/usr/bin/time -v`. **Good result:** `SELF-TEST: PASS`, plus an elapsed time and peak memory figure. This is the estimator validating itself, independent of anything this bundle generates – if this fails, stop and fix the environment before going any further, since nothing downstream can be trusted.

Laptop comparison: PASS. (This laptop is macOS, so `/usr/bin/time -v` isn't available – peak memory wasn't measured locally for this stage; it will be measured for you on the Linux Workbench session.)

4.3 `python scripts/check_machine.py`

Writes `results/machine.json`. Reads the actual memory ceiling from `/sys/fs/cgroup/memory.max` (or the cgroup v1 equivalent) and takes the smaller of that and `/proc/meminfo`'s `MemAvailable`, because on a shared host `free -g` can report the *physical machine's* total RAM, not your session's actual allocation – a common way to get a false sense of headroom. **Good result:** “Usable RAM for this session” roughly matches the memory profile you picked in step 3. If it's far lower, your cgroup is capping you below what you expected – restart the session with a bigger profile before proceeding.

Laptop comparison: 24.0 GiB (via a macOS-only fallback, since there's no cgroup on macOS – on Workbench this number will come from the real cgroup-aware logic).

4.4 The scale ladder: all 0.001 -> all 0.01 -> gate 0.1 -> all 0.1 if GO -> generate 1.0 / validate 1.0 -> gate 1.0

`scripts/run_stage.sh <stage> <scale>` where stage is `generate`, `validate`, `schema-check`, `gate`, `pipeline`, or `all` (all five in order). Scale is a fraction of the real 20% sample (211,593,776 raw rows at scale 1.0).

```
bash scripts/run_stage.sh all 0.001
```

Good result: `generate` and `validate` both exit 0 (`validate` prints “Status: PASS”); `schema-check` prints “missing_required_columns”: []; `gate` prints `DECISION: GO`; `pipeline` launches in the background (see section 5) and eventually produces `run_outputs.json` under `../didi_data/runs/workbench_synthetic_0.001_seed42/` (and two more run directories with `__lag1_common_2014_2018` / `__lag3_common_2014_2018` suffixes, since `run_lag1_vs_lag3: true`).

Then, mandatorily, before trusting anything larger:

```
python scripts/compare_preflight.py --run ../didi_data/runs/workbench_synthetic_0.001_seed42 --scale 0.0
```

At this very small scale, most of the 20 largest real strata will show as `NOT_PRESENT_AT_THIS_SCALE` (their real treated counts are in the low hundreds, so 0.1% of that rounds to 1-3 people – too few for a meaningful score-spread comparison). That is expected, not a failure – see `LOCAL_TEST_REPORT.md` for what this bundle actually measured at 0.001, and move on to 0.01, where the same cells have 10-25 people each and the comparison becomes meaningful.

```
bash scripts/run_stage.sh all 0.01
```

```
python scripts/compare_preflight.py --run ../didi_data/runs/workbench_synthetic_0.01_seed42 --scale 0.0
```

This **must show PASS** (every present stratum within 15 percentage points of the real admissible share, with overlapping score ranges) before you go further. If it does not, see `BUILD_LOG.md`’s calibration-iteration log for what this bundle’s own author found and changed; if you have edited `scripts/generate_raw.py` yourself, this is the check that tells you whether your edit helped or hurt.

Laptop comparison: at 0.01, `LOCAL_TEST_REPORT.md` records this bundle’s own measured admissible-share numbers and PASS/FAIL verdict – compare your Workbench run’s numbers against those as a sanity check, not an exact-match requirement (both runs are stochastic draws from the same generator, seeded the same way, so they should be close but not bit-identical if you’re on a different Python 3.9.x patch, since floating-point library internals can differ subtly across patches).

```
bash scripts/run_stage.sh gate 0.1
```

If `DECISION: GO`:

```
bash scripts/run_stage.sh all 0.1
```

```
python scripts/compare_preflight.py --run ../didi_data/runs/workbench_synthetic_0.1_seed42 --scale 0.1
```

If `DECISION: NO-GO`: that is a **valid, useful result** – it means this Workbench session’s memory profile cannot safely run a synthetic pipeline at 10% of the real sample. Do **not** raise `optimal_match_sparse_max_edges` above 100,000,000, and do not lower `MEMORY_GATE_FRACTION` in `bundle_settings.env` to force a `GO` – both defeat the purpose of the gate. Instead, either stop here (0.01 already gives you a calibration-validated, code-path-exercising result) or restart the session with more memory if a larger profile is available to you.

The full 211,593,776-row **raw generation** at scale 1.0 is a disk-and-time job, not a memory-risk one – it streams to disk at well under 1 GB of peak Python memory regardless of scale (see `BUILD_LOG.md` for the measured per-row generation rate). It should succeed on any session:

```
bash scripts/run_stage.sh generate 1.0
```

```
bash scripts/run_stage.sh validate 1.0
```

Then check whether a full-scale **pipeline run** is safe on this session:

```
bash scripts/run_stage.sh gate 1.0
```

The real 20% data has already passed the panel build and matching stages on the secure machine, so a full-scale *synthetic* pipeline run is a realistic goal on a sufficiently large Workbench session – **provided the 0.01-scale (or larger) preflight comparison above already passed**. If `gate 1.0` says NO-GO, report that honestly; it is real information about what this session can and cannot do, not a problem to be engineered around.

What the full-scale synthetic run is actually for: not proving a full run is *possible* (the real data already answered that question for the panel-build and matching stages) but **timing the stages that have never run at this scale**: outcome bootstraps, longitudinal mediation with the full 200 replicates, MSK dropout sensitivity, the lag1-vs-lag3 comparison, and figure generation – at the real matched-pair count (~115,000 pairs) – and catching any crash (such as the Matplotlib `xerr` figure failure described in section 9) before it costs a day of secure-machine time that can’t be quickly recovered. The stage-by-stage timings this run produces, scaled by core-count ratio to the secure machine, are the numbers you actually need for planning that run’s calendar time.

5. Watching long jobs

`scripts/run_stage.sh pipeline <scale>` launches the pipeline with `nohup` in the background and starts `scripts/monitor_memory.py` alongside it, sampling the *whole process tree* (the pipeline plus every mediation-bootstrap worker it forks) every 10 seconds.

- **Check overall status:** `bash scripts/status.sh` – lists every stage run so far, its exit code, elapsed time, and (for pipeline stages) sampled memory peak, and tells you if a pipeline is still running.
- **Tail the live log:** `tail -f results/pipeline_<scale>.stdout results/pipeline_<scale>.stderr`
- **Resume a dead generate stage:** if a Workbench session dies mid-generation (idle timeout, browser crash, etc.), just re-run the exact same `bash scripts/run_stage.sh generate <scale>` command – `generate_raw.py` skips any shard file whose SHA-256 already matches what it would have produced, so you only regenerate what’s missing.
- **A pipeline stage cannot resume mid-run** – if it dies partway through, re-launch it from scratch with `run_stage.sh pipeline <scale>` (it’s the raw generation, not the pipeline run itself, that’s designed to be interrupted safely).
- **Idle sessions may be suspended by Workbench.** Keep the browser tab open during a long pipeline run, or – if your Workbench admin has enabled it – launch the same command as a Workbench **Job** instead of a Session, which is designed to keep running unattended.

6. The memory rule

`scripts/memory_gate.py` computes a projected peak memory for a target scale and refuses to let `run_stage.sh pipeline` launch if that projection exceeds `MEMORY_GATE_FRACTION` (default 0.8, i.e. 80%) of `results/machine.json`’s `available_ram_gib`. It projects from the **largest completed local run’s own sampled peak**, scaled linearly by row count; if no local run has completed yet, it falls back to the Gate 2 reference points below (from the *old*, uncalibrated generator – see the caveat in `calibration_20pct.json`).

Scale	Rows	Gate 2 reference peak (old generator)	This bundle’s local (laptop) measured peak	Your Workbench measured peak
0.1% (Gate 1)	211,594	1.50 GiB (process RSS)	(see <i>LO-CAL_TEST_REPORT.md</i>)	
1% (Gate 2)	2,115,938	3.87 GiB (process RSS) / 13.93 GiB (whole container, 8 workers)	(see <i>LO-CAL_TEST_REPORT.md</i>)	

Scale	Rows	Gate 2 reference peak (old generator)	This bundle's local (laptop) measured peak	Your Workbench measured peak
10% (Gate 2 projection)	21,159,378	139.3 GiB (projected only – NOT run; NO-GO on a 125.8 GiB VM)	<i>(fill in if you run this)</i>	
100% (full)	211,593,776	~1.39 TiB (projected only – NOT run)	<i>(fill in if you run this)</i>	

Fill in the blank Workbench column yourself as you go – `bash scripts/status.sh` shows you the sampled peak for every pipeline stage you’ve run.

7. Where outputs land

<code>../didi_data/</code>	(DATA_ROOT, next to the bundle folder)
<code>raw_<scale>/</code>	raw synthetic parquet shards + <code>generation_manifest.json</code>
<code>runs/</code>	
<code>workbench_synthetic_<scale>_seed42/</code>	<code>run_outputs.json</code> , figures/, diagrams/
<code>workbench_synthetic_<scale>_seed42__lag1_common_2014_2018/</code>	restricted-window lag-1 spec
<code>workbench_synthetic_<scale>_seed42__lag3_common_2014_2018/</code>	lag-3 comparison spec

Inside `didi_workbench_bundle/results/`: `machine.json`, `gate_<scale>.json` – what the memory gate decided and why - `config_<scale>.yaml`, `config_<scale>_diff.json` – the exact YAML this run used, and what differs from `USER_CONFIG` - `preflight_comparison_<scale>.csv` – the Step-3 calibration check’s per-stratum results - `WORKBENCH_REPORT.md` – one section per scale, built by `scripts/make_report.py` - `<stage>_<scale>.stdout` / `.stderr` / `.time` / `.exit` – raw output for every stage run

Open first, after any run: `results/WORKBENCH_REPORT.md` for the plain-language summary, then the run directory’s `run_outputs.json` for the full structured result.

8. How to change things

All of it lives in two files – never edit `scripts/*.py` or `psm_260915.py` itself:

- **bundle_settings.env:**
 - `PYTHON_CANDIDATES` – add a path if your Workbench’s 3.9 interpreter lives somewhere unexpected. Re-run `scripts/setup_env.sh` after changing.
 - `DATA_ROOT` – where generated data/runs land. Re-run any stage after changing (it just changes where things go, nothing needs re-running unless you want it in the new location).
 - `SCALE_LADDER` – documents the intended progression; `run_stage.sh` accepts any scale you type, this is just a note-to-self field.
 - `SEED` – change for a different (still deterministic) synthetic draw. Re-run `generate`.
 - `SHARDS` – number of raw parquet shards. Re-run `generate`.
 - `BOOTSTRAP_REPLICATES_SMOKE` / `_PRODUCTION` – re-run `make_config.py --scale <s>` (optionally `--production`) then `schema-check/pipeline` for that scale.
 - `MEDIATION_WORKERS` – 0 means auto-pick from `machine.json`; set a specific integer to override once you know the actual per-worker mediation panel size from a completed run’s log line ("mediation panel %.2f GB in memory per worker"). Re-run `make_config.py`.

- `MEMORY_GATE_FRACTION` – how much headroom the gate insists on. Re-run `gate`.
 - **requirements.txt** – pinned package versions. Only touch this if a pin genuinely cannot install on your Workbench Python 3.9.x patch; re-run `scripts/setup_env.sh` afterward, and note in your own log that you deviated from the validated pin set.
-

9. Troubleshooting

- **A pin fails to install:** `setup_env.sh` stops and names the exact failing line. Do not let it silently substitute a different version – fix the underlying issue (often a missing system build dependency for a package with no prebuilt wheel for this platform) or accept that this pin genuinely cannot be satisfied here and say so.
 - **No 3.9.x interpreter found:** see section 3, step 4.
 - **A ^M / carriage-return error running a .sh script:** every file in this bundle was written with Unix line endings (LF only) and verified with a check that no file contains `\r\n` (see `BUILD_LOG.md`). If you see this error, you likely edited a script on Windows with a tool that reintroduced CRLF line endings – re-save it with LF endings only.
 - **Exit code 137 on a pipeline stage:** this is the OOM-kill signature (the Linux kernel’s out-of-memory killer terminated the process). `scripts/make_report.py` flags this automatically. It means the memory gate’s projection was too optimistic for this run – do not retry at the same scale without first checking `results/pipeline_<scale>_memory_samples.txt` for how close the run got and reconsidering the scale, not the gate’s cap.
 - **Matplotlib ValueError: 'xerr' must not contain negative values:** a known, previously-documented failure (`export_20260911/GATE2_REPORT.md`) in mediation/pretrend plotting, which happens when a percentile-bootstrap CI does not contain its own point estimate – a valid statistical outcome, not bad input. This exact copy of `psm_260915.py` already contains a fix for it (see `calibration_20pct.json`’s `estimator.known_matplotlib_xerr_issue`), so it should not recur here – but `scripts/make_report.py` scans for it anyway on every run. **If you see it anyway, report it plainly; do not patch the estimator.**
 - **Expected warnings, not failures** (per Gate 2’s own warning inventory, and reproduced by this bundle’s local test – see `LOCAL_TEST_REPORT.md`): Polars streaming sink/collect fallback messages, and g-computation extrapolation-cap truncation warnings (these can be numerous – Gate 2’s 1% run logged 8,237 of them from a handful of underlying truncated simulated draws). Any warning that is *not* one of these two types, seen in your Workbench run but not in `LOCAL_TEST_REPORT.md`’s inventory, is worth a closer look.
-

10. Appendix

`bundle_settings.env` (full text as shipped)

```
PYTHON_CANDIDATES=/opt/python/3.9.1/bin/python3:/opt/python/3.9.*/bin/python3:/opt/homebrew/bin/python3
DATA_ROOT=../didi_data
SCALE_LADDER=0.001 0.01 0.1 1.0
SEED=42
SHARDS=22
BOOTSTRAP_REPLICATES_SMOKE=20
BOOTSTRAP_REPLICATES_PRODUCTION=200
MEDIATION_WORKERS=0
MEMORY_GATE_FRACTION=0.8
```

(Full commented version, explaining each key, is in the file itself. The `/opt/homebrew/bin/python3.9` candidate is harmless on Workbench – it simply won’t exist there and gets skipped – it was added so this bundle’s own author could test on a laptop.)

requirements.txt (full text as shipped)

```
polars==0.20.16
numpy==1.25.0
pandas==2.3.3
scikit-learn==1.5.2
scipy==1.13.1
pyarrow==15.0.2
matplotlib==3.8.4
PyYAML==6.0.2
psutil>=5.9
```

results/config_0.001.yml (this bundle's own local 0.1%-scale config, as an example)

This is what `scripts/make_config.py --scale 0.001` produced on the laptop that built this bundle. **raw_glob/output_dir** below are **that laptop's own absolute paths** – your Workbench run will produce a config with the same structure but different (Workbench) absolute paths, computed fresh by `make_config.py` from your own `DATA_ROOT`. Everything else (every `PipelineConfig` field's value) will be identical, since it all traces to the same `calibration_20pct.json`-recorded `USER_CONFIG`, not to anything laptop-specific.

```
caliper: 0.01
control_pool: "not_yet_divorced"
controls_per_treated: 1
seed: 42
strict_main_run: true
run_lag1_vs_lag3: true
protect_matching: true
total_rebuild: true
propensity_max_iter: 1000000
propensity_tol: 0.0001
propensity_solver: "lbfgs"
max_propensity_fit_rows: 500000
propensity_score_batch_size: 100000
smd_threshold: 0.1
optimal_match_dense_max_entries: 5000000
generate_figures: true
run_longitudinal_mediation: true
run_msk_dropout_sensitivity: true
gcomp_horizon_years: 5
gcomp_simulation_repetitions: 25
gcomp_bootstrap_simulation_repetitions: 25
gcomp_use_observation_ipcw: true
gcomp_observation_probability_floor: 0.02
gcomp_ipcw_cap: 20.0
gcomp_timevarying_continuous: ["entgelt_value", "rtzb_value"]
gcomp_timevarying_binary: ["non_success_app_this_year", "non_msk_rehab_this_year"]
gcomp_timevarying_categorical: ["income_source_status"]
death_registry_end_year: 2023
msk_diagnosis_codes: [20, 53, 56]
mental_health_diagnosis_codes: [30]
outcome_logit_max_iter: 5000
gcomp_logit_max_iter: 20000
gcomp_logit_tol: 0.000001
gcomp_continuous_log_cap_headroom: 3.0
```

```

year_start: 2012
year_cap: 2018
followup_years: 5
mortality_horizon: 10
lag_depth: 1
lag_comparison_common_year_start: 2014
minimum_category_frequency: 10
data_profile: "canonical"
source_has_entgelt: true
first_marriage_source: "date_fields"
exact_covariates: ["lag1_ge_cat", "lag1_age_band", "lag1_ever_rehab_to_date"]
propensity_numeric: ["lag1_age", "lag1_cum_rehabs_by_year", "lag1_cum_mental_health_rehabs_by_year", "lag1_ever_rehab_to_date"]
propensity_binary: ["lag1_ever_rehab_to_date", "lag1_ever_mental_health_rehab_to_date", "lag1_non_success_rehab_to_date"]
propensity_categorical: ["lag1_whot_bland_cat", "lag1_occ_ll_cat", "lag1_income_source_status", "lag1_financial_status"]
raw_glob: "<your DATA_ROOT>/raw_0.001/raw_*.parquet"
output_dir: "<your DATA_ROOT>/runs"
sample_tag: "workbench_synthetic_0.001_seed42"
outcome_bootstrap_replicates: 20
absolute_effect_bootstrap_replicates: 20
mediation_bootstrap_replicates: 20
mediation_bootstrap_workers: 2
optimal_match_sparse_max_edges: 100000000
paper_reporting:
  minimum_cell_people: 20
  pool_small_diagnosis_codes: true
  protect_matchability_cells: true
  withhold_small_overlap_counts: true

```